PAO — Product Requirements Document

Oct 9, 2026 · @Sakib Ahmed · Draft v0.1

1. Product overview

PAO is a mobile marketplace for Bangladesh. It connects customers with verified, independent service providers near them, such as electricians, plumbers and AC technicians. Customers can book a provider at a fixed, platform-set price.

Item	Value
Product / app name	PAO
Internal project codename	Lift (see Open questions)
Category	Helping services / home services marketplace
Tagline options	"Need it? PAO it" · "Find. Book. Done."
Domains to buy	getpao.com and pao.bd (buy both if the name is final)
Platforms	Customer app and Provider app (Android + iOS, Flutter); Admin web panel
Backend	NestJS (TypeScript), API-first, modular monolith built to split into microservices later
First release	AI-built MVP with 4–5 services, then handed to human engineers

Purpose of this document. It is the single source of truth for building the MVP. It is written for two readers: the AI coding agent that builds the first version, and the engineers who take it over afterwards. Handwritten notes still to come will be merged in (section 13).

2. Problem and concept

PAO applies the Uber/Pathao model to home services. Instead of employing technicians, it lets independent providers who are already working nearby sign up, get verified and receive bookings.

Reference point. Sheba.xyz offers similar services (electrician, plumber and so on) in Bangladesh, but it delivers them through its own employees. That approach limits how far it can scale and how many areas it can cover.

PAO's approach.

- Open supply. Any qualified local provider can enrol through the Provider app.
- Proximity. Customers see providers near their location, not a central dispatch.
- Choice. Customers compare providers by rating and choose one themselves.
- Fixed pricing. PAO defines every service name and price, so there is no haggling and every provider charges the same for the same job.
- Trust as the core product. Providers enter people's homes, so verification is mandatory. Without it the platform cannot operate (section 6).

Who benefits.

Party	Problem today	What PAO gives them
Customer	Hard to find a reliable, safe worker quickly; prices are unclear	Verified nearby providers, ratings, fixed prices, booking in a few taps
Provider	Depends on word of mouth; irregular work	Steady job requests from nearby customers, a public rating, a verified badge
PAO	—	Platform revenue (model to be decided, see section 13)

3. Users, roles and applications

PAO ships as three separate clients over one backend API. You asked for separate customer and provider apps. The admin panel is a derived requirement: fixed service prices and provider verification both need an internal tool to manage them.

Application	Platform	Used by	Main job
PAO (Customer app)	Flutter, Android + iOS	Customers	Browse services, see nearby providers, book, track, pay, rate
PAO Partner (Provider app)	Flutter, Android + iOS	Service providers	Enrol, submit verification documents, go online/offline, accept or reject bookings, complete jobs, see earnings
PAO Admin	Web (Flutter Web or a separate web stack, to be decided)	PAO operations staff	Manage the service catalog and prices, review verifications, manage users, handle complaints, monitor bookings

Roles.

Customer. Any registered user with a verified phone number.
Provider. A user who enrolled through the Provider app. A provider can receive bookings only after passing at least Level 1 verification.
Verifier / Ops agent (admin role). Reviews documents and runs field verification.
Catalog manager (admin role). Creates and edits services and prices.
Super admin. Manages admin users and permissions.

One person may hold both a customer and a provider account. The two accounts share an identity record, but each app uses only its own role.

4. Service catalog

PAO launches with 4–5 services and must be able to add more later without code changes. To make that possible, each service is tagged with a service model, because the services in your list do not all book the same way.

Service models.

Model	How it works	Price shape
On-demand job	One visit to fix or install something; the provider accepts and arrives	Fixed price per job or per unit (e.g. per AC unit)
Duration hire	A provider is engaged for a period of time	Fixed price per period (day / month / 3 months / 6 months…)
Listing	PAO shows items, not people; the booking price; possibly a listing customer contacts the owner	No or lead fee
Partner referral	PAO passes the customer to a partner company	Partner's own package price; PAO may earn a referral fee

Full service list from your notes.

Service	Model	Notes
Electrician	On-demand job	Strong MVP candidate
Plumber	On-demand job	Strong MVP candidate
AC service	On-demand job	Cleaning, gas refill, repair; priced per unit
AC maker / installer	On-demand job	Could merge with AC service as sub-services
Home salon	On-demand job	Provider visits the home; women providers may be required for female customers
Driver	Duration hire	1 day / 1 month / 3 months / 6 months / custom
Home worker	Duration hire	Domestic help; needs the strictest verification (lives or works inside the home daily)
Find flat	Listing	Different product (property listings); recommend a later phase
Internet provider	Partner referral	ISP packages via partners; later phase
"Find …" (more)	Any	The catalog must allow new categories from the admin panel

Proposed MVP set (5 services, to confirm). Electrician, Plumber, AC service,

Home salon,
  and Driver (1-day hire only). This covers the on-demand model fully and tests the
  duration-hire model in its simplest form. Flat listing and internet referral are deferred

because they are different products.

Catalog structure. Category → Service → Sub-service (the bookable item with a fixed price). For example: Electrical → Electrician → "Ceiling fan installation", ৳X per fan. Admins create and edit the catalog, and every price change is versioned so past bookings keep the price that applied when they were made.

5. Core booking flow and business rules

In PAO, the customer chooses the provider and the provider accepts or rejects. The platform never assigns a job automatically in the MVP.

Happy path (on-demand job).

1. The customer opens the app, which detects their location or lets them pick a saved address.
2. The customer taps a service and picks a sub-service and quantity. The fixed price is shown.
3. The app lists verified, online providers within the search radius, sorted by distance and rating. Each card shows the provider's name, photo, verification badge, rating, number of completed jobs and distance.
4. The customer opens a provider's profile, chooses a time ("as soon as possible" or a scheduled slot) and confirms the booking.
5. The provider receives a push notification and has a time limit to accept or reject.
6. If the provider accepts, the customer sees the confirmation and the provider's contact details. If the provider rejects or the time runs out, the customer is asked to pick another provider.
7. The provider marks "On the way", then "Arrived". The provider starts the job by entering a 4-digit start code that the customer reads from their app.
8. The provider marks the job complete. Payment is settled (cash in the MVP), and both sides rate each other.

booking states · 6 main states, 2 exits

If the provider rejects the booking or does not reply in time, the customer goes back to the provider list. Cancellation is allowed until the start code is entered.

Business rules.

Rule	Requirement
Pricing	Service names and prices are set only by PAO admins. Providers cannot change prices. The price is copied onto the booking when it is created.
Extra work	If the job needs more than was booked, the provider adds catalog items in the app and the customer approves them. Free-text prices are not allowed.
Provider visibility	Only providers who are verified (Level 1 or higher), online, registered for that service and within the radius appear in search.
Search radius	Configurable per service (default value to be decided, e.g. 5 km).
Accept time limit	Configurable (e.g. 3 minutes for ASAP bookings, longer for scheduled ones).
One active job	In the MVP, a provider cannot accept a new ASAP booking while another job is in progress.

Rule	Requirement
Cancellation	Free before the provider accepts. After acceptance, cancellations are recorded, and a fee or penalty policy is to be decided. Repeated provider cancellations lower their ranking.
Ratings	1–5 stars plus an optional comment, available only after a completed job. Customers rate providers and providers rate customers.
Disputes	Either party can report a problem on a booking. It goes to the admin complaint queue.

Duration hire (driver). The flow is the same, but the customer picks a period instead of a quantity, and the booking has a start date and an end date. Multi-day hires show a daily check-in in later phases.

6. Verification, trust and safety

No provider can receive a booking without passing verification. Providers enter customers' homes, so verification is a launch blocker, not a feature to add later.

6.1 Verification levels

Level	Name	How it is earned	Badge shown to customers	Can receive bookings
0	Registered	Signed up and verified their phone by OTP	None	No
1	Document Verified	All Level 1 documents submitted and approved by a PAO verifier	"Verified"	Yes
2	Skill Verified	Level 1, plus PAO has inspected their work in person (skill test or supervised job)	"PAO Pro"	Yes, ranked higher

Services with in-home, unsupervised access (home worker, and driver for long hires) may require Level 2 before the first booking. This can be set per service in the admin panel.

6.2 Level 1 — document verification

The provider submits the following in the Provider app. A verifier reviews each item in the admin panel and approves or rejects it with a reason.

#	Item	Purpose	Check performed
1	Mobile number (OTP)	Identity anchor, contact	Automatic OTP check; one account per number
2	National ID (NID), front and back	Legal identity	NID number format check; data matched against the official NID verification service (provider to be chosen); name and date of birth must match the profile
3	Live selfie	Proves the person is the NID holder	Liveness check plus face match against the NID photo; a verifier confirms
4	Police clearance certificate	Criminal background	Verifier checks the certificate's authenticity and date (e.g. issued within the last 12 months)
5	Present and permanent address	Traceability	Verifier review; optional utility bill as proof
6	Emergency contact / guarantor	Accountability	Name, relationship and phone number; the phone number is verified by call or OTP
7	Skill or trade proof (if	Competence	Training or trade certificate,

any) or work photos. Optional for Level 1, used for Level 2

#	Item	Purpose	Check performed
8	Service area and categories	Matching	Provider selects services and a home location; admin confirms categories match their proof
9	Agreement to the provider code of conduct	Legal	In-app acceptance, stored with a timestamp and version

6.3 Level 2 — skill and field verification

Level 2 adds an in-person check of the provider's actual work on top of their documents.

PAO schedules an in-person interview or skill test, at a PAO location or a supervised real job.

The verifier records the outcome in the admin panel using a per-service checklist (for example, an electrician checklist covers safe wiring, tool use and earthing awareness).

Optional: a physical address visit, recorded with photos and the GPS position.

Result: pass, fail or retest. A failed provider stays at Level 1 and may retry after a cooling-off period.

6.4 Ongoing trust

Re-verification. The police clearance certificate expires every 12 months and NID data is re-checked if the profile changes. An expired document suspends bookings automatically until it is renewed.

Quality thresholds. A rating below a set floor (e.g. 3.5 after 10 jobs), repeated cancellations or a verified complaint triggers review or suspension.

Audit trail. Every verification decision stores who decided, when, what was decided and why. These records cannot be edited.

Account status. Pending → Active → Suspended → Banned. Banned identities (NID, phone, face) cannot re-register.

6.5 Customer-side safety features

Feature	Phase
Customer phone OTP verification (also protects providers)	MVP
Provider photo, name, badge and rating shown before and after booking	MVP
4-digit start code: the job cannot start until the provider enters the customer's code	MVP
In-app call through a masked number, so phone numbers are not exposed	Phase 2
Share live job status with a family member	Phase 2
SOS button (alerts PAO operations and the emergency contact)	Phase 2
Preference for a woman provider (e.g. home salon)	MVP for salon

6.6 Handling of verification data

NID images, selfies and police certificates are sensitive personal data.

They are stored in private object storage and accessed only through short-lived signed URLs.

They are encrypted at rest and visible only to admin roles with the verification permission. Every view is logged.

They are never returned by any customer-facing API. Customers see only the badge, never the documents.

Retention: documents are kept while the account is active, plus a fixed period after closure (to be decided in line with Bangladeshi law).

7. Feature list

Everything marked MVP is in scope for the AI-built first release. Phase 2 and Later items are listed so the architecture leaves room for them.

7.1 Customer app

ID	Feature	Phase
C-01	Sign up / log in with phone + OTP; basic profile (name, photo optional)	MVP
C-02	Location detection and saved addresses (home, office, other) with a map pin	MVP
C-03	Home screen: service categories, search, banner area	MVP
C-04	Service detail: sub-services with fixed prices, quantity selector, what's included	MVP
C-05	Nearby provider list: distance, rating, jobs done, badge; sort by distance or rating	MVP
C-06	Provider profile: photo, bio, services, badge level, rating breakdown, reviews	MVP
C-07	Booking: ASAP or scheduled slot, address, note to provider, price summary, confirm	MVP
C-08	Booking status tracking (requested → accepted → on the way → arrived → in progress → completed)	MVP
C-09	Start code display and approval of extra items	MVP
C-10	Cancel booking with reason	MVP
C-11	Rate and review provider	MVP
C-12	Booking history and booking detail	MVP
C-13	Push notifications for every status change	MVP
C-14	Report a problem on a booking	MVP
C-15	Language: Bangla and English	MVP
C-16	Live map tracking of the provider on the way	Phase 2
C-17	Digital payment (bKash, Nagad, card)	Phase 2
C-18	Masked in-app calling, chat with provider	Phase 2

ID	Feature	Phase
C-19	Favourite providers, rebook	Phase 2
C-20	SOS and share trip status	Phase 2
C-21	Promo codes, referral	Later
C-22	Flat listings, internet packages	Later

7.2 Provider app

ID	Feature	Phase
P-01	Sign up with phone + OTP	MVP
P-02	Enrolment wizard: profile, services offered, service area, documents, selfie, emergency contact, code of conduct	MVP
P-03	Verification status screen with per-document status and rejection reasons; re-upload	MVP
P-04	Online / offline toggle (location shared only while online)	MVP
P-05	Incoming booking request with countdown; accept / reject with reason	MVP
P-06	Active job flow: on the way → arrived → enter start code → add extra items → complete	MVP
P-07	Navigation hand-off to Google Maps for directions	MVP
P-08	Job history and earnings summary (daily, weekly, monthly)	MVP
P-09	Rate customer	MVP
P-10	Profile, ratings and reviews received	MVP
P-11	Document expiry reminders	MVP
P-12	Push notifications	MVP
P-13	Language: Bangla and English	MVP

ID	Feature	Phase
P-14	Weekly availability schedule	Phase 2
P-15	Commission statements, payouts to bKash/bank	Phase 2
P-16	Level 2 test booking from the app	Phase 2

7.3 Admin panel

ID	Feature	Phase
A-01	Admin login with roles and permissions	MVP
A-02	Catalog management: categories, services, sub-services, prices, service model, required verification level, search radius	MVP
A-03	Verification queue: review each document, approve or reject with reason, assign Level 1	MVP
A-04	Level 2 scheduling and checklist recording	MVP
A-05	Provider and customer management: search, view, suspend, ban	MVP
A-06	Bookings monitor: list, filter, booking timeline	MVP
A-07	Complaints queue	MVP
A-08	Audit log viewer	MVP
A-09	Basic dashboard: providers by status, bookings per day, completion rate	MVP
A-10	Platform settings (timeouts, rating thresholds, cancellation rules)	MVP
A-11	Push notification broadcasts	Phase 2
A-12	Finance: commissions, payouts, reconciliation	Phase 2

8. Screen inventory

The MVP needs about 22 customer screens, 20 provider screens and 12 admin screens.

Design all of them before any development starts. Each row below names the screen and what it must contain, and the feature IDs link back to section 7.

8.1 Customer app

#	Screen	Key contents	Features
1	Splash	Logo, tagline	—
2	Onboarding (3 slides)	Find nearby help, verified providers, fixed prices; language choice	C-15
3	Phone entry	+880 number field, terms link	C-01
4	OTP	6-digit input, resend timer	C-01
5	Profile setup	Name, optional photo	C-01
6	Location permission / pick address	Map pin, address search, label	C-02
7	Home	Current address, search bar, category grid, active-booking banner	C-03
8	Search results	Matching services and sub-services	C-03
9	Service detail	Sub-service list with prices, quantity, inclusions/exclusions, Continue	C-04
10	Provider list	Provider cards (photo, name, badge, rating, jobs, distance), sort, empty state	C-05
11	Provider profile	Header, badge explanation, services, rating breakdown, reviews, Book button	C-06
12	Booking setup	ASAP / schedule picker, address, note, price summary, payment = cash	C-07
13	Waiting for provider	Countdown, cancel option, "choose another provider" on timeout or reject	C-07, C-10

14	Booking active	Status stepper, provider card, call button, start code, cancel	C-08, C-09

#	Screen	Key contents	Features
15	Extra items approval	Added items with fixed prices, new total, approve / decline	C-09
16	Job completed	Final bill, pay cash confirmation	C-08
17	Rate provider	Stars, tags, comment	C-11
18	Bookings list	Upcoming / past tabs	C-12
19	Booking detail	Timeline, bill, provider, report problem	C-12, C-14
20	Report a problem	Reason list, description, photo	C-14
21	Notifications	List of updates	C-13
22	Account / settings	Profile, addresses, language, help, terms, log out, delete account	C-02, C-15

8.2 Provider app

#	Screen	Key contents	Features
1	Splash / onboarding	What PAO Partner offers, language	P-13
2	Phone entry + OTP	Same as customer	P-01
3	Enrolment: personal info	Name, date of birth, gender, present/permanent address	P-02
4	Enrolment: services	Pick categories and services offered, experience in years	P-02
5	Enrolment: service area	Home location pin, working radius	P-02
6	Enrolment: NID	Capture front and back, NID number	P-02
7	Enrolment: selfie	Guided live selfie	P-02
8	Enrolment: police clearance	Upload certificate, issue date	P-02

9	Enrolment: skill proof	Certificates, work photos (optional)	P-02
10	Enrolment: emergency contact	Name, relation, phone	P-02

#	Screen	Key contents	Features
11	Code of conduct	Text, accept checkbox	P-02
12	Verification status	Per-item status, rejection reasons, re-upload, Level 2 info	P-03
13	Home (dashboard)	Online toggle, today's earnings, active job card, badge level	P-04
14	Incoming request	Service, items, price, distance, address area, countdown, accept / reject	P-05
15	Active job	Status actions, customer name, address, navigate, call	P-06, P-07
16	Enter start code	4-digit input	P-06
17	Add extra items	Catalog item picker for that service, total	P-06
18	Complete and rate customer	Final bill, cash received, stars	P-06, P-09
19	Jobs and earnings	History list, totals by period	P-08
20	Profile and reviews documents	Public profile preview, reviews, and expiry	P-10, P-11

8.3 Admin panel

#	Screen	Key contents	Features
1	Login	Email + password, 2FA	A-01
2	Dashboard	Key counts and charts	A-09
3	Catalog tree	Categories → services → sub-services	A-02
4	Service editor	Name (BN/EN), icon, model, price, required level, radius	A-02
5	Verification queue	Pending providers, filters, age of request	A-03

6	Verification review	Documents side by side, face match result, approve / reject per item	A-03

#	Screen	Key contents	Features
7	Level 2 session	Schedule, checklist, result	A-04
8	Providers list and detail	Status, level, history, suspend / ban	A-05
9	Customers list and detail	Bookings, reports, suspend	A-05
10	Bookings monitor and detail	Filters, timeline of state changes	A-06
11	Complaints	Queue, assign, resolve	A-07
12	Settings and audit log	Platform settings, admin users, audit entries	A-08, A-10

8.4 Page map — Main app (customer)

Main app (customer) page map · 4 tabs, 27 pages

Tapping a category on Home skips Search results and goes straight to Service detail. If the provider rejects the booking or does not reply in time, Waiting for provider sends the customer back to Provider list. Account now includes Edit profile, Saved addresses, Add/edit address and Help & support as their own pages.

8.5 Page map — Merchant app (service provider)

Merchant app (service provider) page map · 4 tabs, 24 pages

Until Level 1 is approved, the merchant only sees the Verification page. From there they can re-upload any rejected document and repeat the relevant enrolment step. After approval, an expired document sends them back to Verification and hides incoming requests until it is renewed.

9. Technical architecture

The backend is a NestJS (TypeScript) modular monolith, deployed as one Node image with three entry points: api, worker and migrate. It is built so that any module can later become its own microservice by moving its folder and replacing in-process calls with network calls. The two Flutter apps talk to it only through a versioned REST API.

9.1 Stack

Layer	Choice	Notes
Backend language	NestJS (TypeScript, Node.js 22)	REST/JSON API, OpenAPI 3 spec written first
Mobile apps	Flutter (Android + iOS)	Two apps in one monorepo, sharing packages
Admin panel	To be decided: Flutter Web (reuses packages) or a separate web framework	See section 13
Database	PostgreSQL + PostGIS	PostGIS powers the "providers within X km" search
Cache / rate limiting, online-provider limits	Redis (can be deferred; Postgres is enough for the MVP)	OTP rate cache
File storage	S3-compatible object storage, private buckets	Verification documents, profile photos
Push notifications	Firebase Cloud Messaging	Both apps
SMS / OTP	Bangladeshi SMS gateway (to be chosen)	Behind an interface so it can be swapped
Maps	Google Maps SDK (Barikoi as a local alternative)	Behind an interface
Packaging	Docker; one container for the API plus Postgres	Simple MVP deployment

9.2 Backend modules

Module	Owns	Example public contract methods
identity	Accounts, phone OTP, sessions, JWT, roles	GetAccount , HasRole
customer	Customer profile, saved addresses	GetCustomer , GetAddress
provider	Provider profile, services offered, online status, live location	GetProvider , FindNearby , IsAvailable
verification	Documents, checks, levels, expiry, Level 2 sessions	GetLevel , CanReceiveBookings
catalog	Categories, services, sub-services, versioned prices	GetPriceSnapshot , GetService
booking	Booking lifecycle, state machine, start code, extra items	GetBooking , CountActiveJobs
rating	Reviews and rating aggregates	GetProviderRating
notification	Push, SMS, in-app notifications	Send
media	Uploads, signed URLs	CreateUploadURL , GetViewURL
admin	Admin users, permissions, settings, complaints	GetSetting
audit	Append-only audit log	Record

9.3 Module boundary rules (what makes the later split easy)

1. One contract package per module. Each module has a contract/ folder. Its service file (e.g. booking-service.ts) holds the module's service-to-service interface, which is the only set of methods other modules may call. The same folder holds the request/response DTOs and event definitions (this is the separate service-to-service file you asked for).
2. Import only contracts. Module A may import moduleB/contract and nothing else from module B. This is enforced by a lint rule in CI (dependency-cruiser).
3. Own your data. Each module has its own Postgres schema (booking.* , catalog.* and so on). There are no joins or foreign keys across schemas, and one module never reads another module's tables.

4. Copy, don't reference. A booking stores snapshots (price, service name, provider display name) rather than reading them live from other modules.
5. Events for side effects. Modules publish domain events (e.g. BookingAccepted) on an in-process event bus interface, and other modules subscribe. Events are written through an outbox table so the bus can later be swapped for NATS, RabbitMQ or Kafka without changing module code.
6. No cross-module transactions. A transaction never spans two modules. Workflows that touch several modules use events or explicit compensation.
7. Swappable implementations. In the monolith, the contract interface is implemented by an in-process adapter. In a microservice, the same interface is implemented by an HTTP/gRPC client. Calling code does not change.

9.4 Inside a module (clean architecture + repository pattern)

```
backend/
  src/main.ts                    # composition root: wires modules together
  src/worker.ts, src/migrate.ts  # worker and migration entry points
  src/
    platform/                    # shared infrastructure only: config, db, logger,
                                 # http server, auth guards, event bus, outbox
    modules/
      booking/
        contract/                # PUBLIC: the only folder other modules import
          booking-service.ts     # service-to-service interface (BookingService) + injection token
          dto.ts
          events.ts
        domain/                  # entities, value objects, state machine, rules (no imports from outside)
        app/                     # use cases: create-booking.ts, accept-booking.ts, ...
        port/                    # repository and gateway interfaces used by app/
        adapter/
          postgres/              # repository implementations
          http/                  # controllers, request validation, mapping to use cases
          inproc/                # implementation of contract/ for other modules
        booking.module.ts        # Nest module: wires the module, exports its contract
```

```
     and subscribers
            README.md                  # what the module owns, its contract,
events
     in and out
        migrations/<module>/           # SQL migrations per module schema
        api/openapi.yaml               # API contract for the Flutter apps
        docs/adr/                      # architecture decision records
```

Dependencies point inwards only: adapter → app → domain . The domain layer has no knowledge of HTTP, SQL or other modules. Use cases depend on interfaces in port/ , never on concrete repositories (dependency inversion).

9.5 Flutter structure

```
     pao-mobile/                       # monorepo managed with melos
        apps/
            customer/                  # PAO
            provider/                  # PAO Partner
        packages/
            pao_core/                  # config, networking, auth/session,
error
     handling
         pao_api/                      # API client generated from
api/openapi.yaml
            pao_ui/                    # design system: theme, colours,
typography,
     shared widgets
            pao_l10n/                  # Bangla + English strings
```

Each app is organised feature-first (features/booking/{data,domain,presentation}), using the same clean-architecture layering as the backend. One state-management library is used throughout (recommended: flutter_bloc or Riverpod, decided once and recorded in an ADR).

9.6 API conventions

REST, JSON, versioned under /v1 . Separate route groups: /v1/customer/* , /v1/provider/* , /v1/admin/* .

Authentication: short-lived JWT access token plus a rotating refresh token. Role checks happen in middleware.

An idempotency key is required on booking creation and on any payment call.

One error format everywhere: { "error": { "code", "message", "details" } } , with stable error codes the apps can translate.

Cursor-based pagination on all lists. All times are stored in UTC and shown in Asia/Dhaka.

10. Engineering and documentation standards

The MVP is built by AI but maintained by humans. Every rule below exists so that a new engineer can open the repository and be productive within a day.

Code style: it must read as if a careful human wrote it.

Names carry the meaning: AcceptBooking , providerID , ErrBookingAlreadyAccepted . No generic data , handleStuff or utils2 .
Comments explain why, never what. No comment on every line, no banner comments, no restating the code, no leftover AI notes such as "Here we…" or "// Step 1".
Functions are small and do one thing. A deeply nested function is a sign it needs splitting.
Code follows the standard formatters and linters: Prettier , ESLint , dart format , flutter analyze , with the lint config committed to the repository.
No dead code, commented-out code, or TODOs without an issue reference.

Design principles.

SOLID.
Single responsibility per type and function.
Open/closed through interfaces: a new SMS provider is a new adapter, not edits to existing code.
Liskov: every adapter honours its port's contract.
Interface segregation: small ports.
Dependency inversion: use cases depend on ports.
Repository pattern. All database access goes through repository interfaces in port/ .
SQL lives only in adapter/postgres .
Clean architecture. The layering and boundary rules in sections 9.3 and 9.4 are mandatory, and CI fails the build when they are broken.
Explicit errors. Errors are wrapped with context and mapped to API error codes at the HTTP edge only.

Testing (a task is not done until its tests pass).

Level	What	Minimum
Unit	Domain rules, booking state machine, price calculation, verification level logic	Every rule and every state transition
Integration	Repositories against a real Postgres (testcontainers)	Every repository method
API	Handlers end to end, through the HTTP layer	Every endpoint: success plus main failure cases
Architecture	Import rules between modules	Runs in CI
Flutter	Widget tests for key screens; unit tests for blocs/notifiers	Booking flow and enrolment flow

Documentation that must exist in the MVP.

Root README.md : what PAO is, how to run everything locally with one command, folder map

docs/architecture.md : module map, boundary rules, how to extract a module into a service

README.md in every module: responsibility, contract methods, events published and consumed, tables owned

api/openapi.yaml : complete, with examples, served as interactive docs in development

docs/adr/ : one record per significant decision (modular monolith, PostGIS, state management, auth)

docs/verification.md : the verification process from section 6, as an operations runbook

docs/domain-glossary.md : booking, sub-service, level, start code and other terms

CONTRIBUTING.md : branch naming, commit style (Conventional Commits), PR checklist

Seed data script with the 5 MVP services and sample providers for demos

11. Non-functional requirements

Area	Requirement
Security	HTTPS only; passwords (admin) hashed with argon2/bcrypt; OTP rate-limited per number and IP; admin 2FA; least-privilege admin roles; secrets kept in environment variables, never in the repository
Privacy	Customer exact address shared with provider only after acceptance; provider location tracked only while online or on a job; verification data rules from 6.6; account deletion supported
Performance	Nearby-provider search under 300 ms at p95 for one city; booking request reaches provider in under 3 s
Availability	MVP target 99.5%; daily database backups with a tested restore
Scalability	Stateless API instances so it can scale horizontally; modules extractable as described in 9.3
Localisation	Bangla and English throughout; ৳ (BDT) currency; Asia/Dhaka time display
Low-end devices	The apps work on mid-range Android phones and slow 3G/4G; images are compressed before upload
Observability	Structured JSON logs with request IDs; health endpoint; basic metrics (requests, errors, latency); error tracking in both apps
Accessibility	Readable text sizes, sufficient contrast, tap targets of at least 48 dp

12. AI-built MVP delivery plan

AI writes all of the MVP code. Your role is to review each stage and correct it where it goes wrong. Each stage below ends with something runnable and tested, so mistakes are caught early rather than at the end.

1. Design first. Produce wireframes and then UI designs for every screen in section 8, plus the design system (colours, type, components). Freeze the MVP screen list.
2. Contracts. Write api/openapi.yaml , the module list, each module's contract/ service interface and events, and the database schema per module. Review all of these before any implementation.
3. Backend skeleton. Set up the platform layer, module scaffolding, CI (lint, architecture rules, tests), Docker setup and the root documentation.
4. Identity + catalog + admin basics. OTP login, roles, catalog management and seed data for the 5 MVP services.
5. Provider + verification. Enrolment APIs, document upload, verification queue and levels, expiry jobs.
6. Discovery + booking. Nearby search, the booking state machine, start code, extra items, cancellation and timeouts.
7. Rating + notification + audit. Reviews, push notifications on every booking event, audit log.
8. Flutter apps. Shared packages, then the Provider app (enrolment first), then the Customer app, both against the real API.
9. Admin panel. The verification review screens come first, because the platform cannot onboard providers without them.
10. Hardening and handover. End-to-end test of the full booking flow, a security checklist, a documentation pass, and a handover README for the human team.

How to brief the AI at each stage. Give it this document, the relevant section, the module's README and the contract files. Ask for one module or one feature at a time, together with its tests. Do not accept a stage until tests pass and the module README is updated.

13. Open questions and pending inputs

These decisions are needed before or during the design stage. Each one changes what gets built.

Name. Your note says "Project name: lift" and "App name: PAO". Is Lift the internal codename and PAO the public brand, or is one of them being replaced?

MVP services. Confirm the 5 services: Electrician, Plumber, AC service, Home salon, Driver (1-day)?

Revenue model. Commission per job (what %), provider subscription, customer service fee, or a mix? This decides whether the Finance module is in the MVP.

Payment. Cash only for the MVP, with bKash/Nagad in Phase 2? If a commission is taken from cash jobs, how do providers pay it to PAO?

Launch area. One city first (Dhaka?), and which areas? This sets the search radius and how many providers must be recruited.

Provider recruitment. Self-enrolment only, or will PAO also onboard providers in the field? Field onboarding makes Level 2 easier.

Level 2 requirement. Which services require Level 2 before the first booking (home worker for sure; driver?)?

Cancellation policy. Fees or penalties after acceptance, for both customer and provider.

Admin panel technology. Flutter Web (one codebase) or a separate web framework?

Third-party providers. Which SMS gateway, NID verification service, and maps provider (Google or Barikoi).

Home salon gender rule. Should customers be able to request a woman provider, and should women providers be able to accept female customers only?

Pending documents. Your handwritten notes are still to be uploaded. They will be merged into sections 4–8 when they arrive.

Domains. Buy getpao.com and pao.bd once the name is confirmed.